Лабораторная работа №7 "Создание компонентного интерфейса с использованием React"

Срок сдачи: четверг, 30 апреля 2026, 23:59

1. ЦЕЛЬ РАБОТЫ

Освоение концепций компонентного подхода в React. Практическое знакомство с созданием компонентов, работой с JSX и передачей данных через props. Подготовка компонентной базы для последующего изучения управления состоянием и работы с сервером (в следующем семестре).

2. НЕОБХОДИМЫЕ ЗНАНИЯ

Для успешного выполнения работы студент должен знать:

- Продвинутый JavaScript (ES6+, функции, объекты, массивы)
- HTML5 и CSS3 (семантическая верстка, базовые стили)
- Основы компонентного подхода в веб-разработке
- Работу с API (из ЛР6) - как база для будущей интеграции

3. ЗАДАЧИ РАБОТЫ

- Освоить создание React-приложения с использованием Vite
- Изучить синтаксис JSX и его отличия от обычного HTML
- Научиться создавать функциональные компоненты
- Освоить передачу данных между компонентами через props
- Понять концепции SPA, Virtual DOM и различия между фреймворками
- Создать библиотеку компонентов для последующего использования

ПОДРОБНОЕ ПОШАГОВОЕ ВЫПОЛНЕНИЕ РАБОТЫ

ШАГ 1: ПОДГОТОВКА РАБОЧЕГО ОКРУЖЕНИЯ

Объяснение: Перед началом работы необходимо настроить среду разработки. Мы будем использовать Vite - современный инструмент для создания React-приложений.

```
# 1. Откройте терминал и перейдите в папку ваших проектов
cd ~/projects

# 2. Создайте новое React-приложение с помощью Vite
npm create vite@latest my-react-app -- --template react

# 3. Перейдите в созданную папку проекта
cd my-react-app

# 4. Установите зависимости
npm install

# 5. Установите React Router для будущих работ
npm install react-router-dom

# 6. Запустите приложение в режиме разработки
npm run dev
```

Что происходит: Vite создает базовую структуру React-приложения с горячей перезагрузкой. После выполнения команды `npm run dev` приложение будет доступно по адресу `http://localhost:5173`.

ШАГ 2: ЗНАКОМСТВО СО СТРУКТУРОЙ ПРОЕКТА

Объяснение: React-приложение имеет определенную структуру. Давайте изучим основные файлы и папки:

```
my-react-app/
├── public/                 # Статические файлы
├── src/
│   ├── assets/            # Изображения, шрифты
│   ├── components/        # React-компоненты (создадим)
│   │   ├── ui/           # Базовые UI компоненты
│   │   ├── layout/       # Компоненты layout
│   │   └── features/     # Функциональные компоненты
│   ├── pages/            # (Заготовка для ЛР18 - маршрутизация)
│   ├── data/             # Моковые данные
│   ├── App.jsx           # Главный компонент
│   ├── main.jsx          # Точка входа
│   └── index.css         # Глобальные стили
├── package.json          # Зависимости проекта
└── vite.config.js        # Конфигурация Vite
```

Действия:

1. Создайте папки `components`, `components/ui`, `components/layout`, `components/features`, `pages`, `data`
2. Изучите содержимое `App.jsx` и `main.jsx`

ШАГ 3: СОЗДАНИЕ БАЗОВЫХ КОМПОНЕНТОВ

Объяснение: Компоненты - это строительные блоки React-приложения. Мы начнем с простых презентационных компонентов, которые получают данные через props.

Компонент кнопки (`src/components/ui/Button.jsx`):

```
// Объяснение: Этот компонент демонстрирует базовые принципы React:
// 1. Функциональный компонент - просто функция, возвращающая JSX
// 2. Props - параметры, передаваемые в компонент
// 3. JSX - синтаксис, похожий на HTML, но с возможностью использовать JavaScript

const Button = ({
  children, // Содержимое кнопки (текст или другие элементы)
  onClick, // Функция-обработчик клика
  variant = 'primary', // Вариант стиля (значение по умолчанию)
  type = 'button', // Тип кнопки
  disabled = false, // Состояние disabled
}) => {
  // Возвращаем JSX - описываем, как компонент выглядит
  return (
    <button
      className={`btn btn--${variant}`} // Динамические классы CSS
      type={type}
      onClick={onClick}
      disabled={disabled}
    >
      {children} {/* Отображаем содержимое */}
    </button>
  );
};

export default Button; // Экспортируем компонент для использования в других файлах
```

Компонент карточки (`src/components/ui/Card.jsx`):

```
// Объяснение: Этот компонент показывает, как создавать композитные компоненты
// и работать с условным рендерингом в JSX

const Card = ({
  title,
  description,
  image,
  children, // Особый prop - содержимое между открывающим и закрывающим тегами
  className = '', // Дополнительные CSS классы
}) => {
  return (
    <article className={`card ${className}`}>
      {/* Условный рендеринг: показываем изображение только если оно передано */}
      {image && (
        <div className='card__image'>
          <img src={image} alt={title} />
        </div>
      )}

      <div className='card__content'>
        <h3 className='card__title'>{title}</h3>

        {/* Условный рендеринг описания */}
        {description && <p className='card__description'>{description}</p>}

        {/* children - позволяет вкладывать другие компоненты */}
        {children && <div className='card__children'>{children}</div>}
      </div>
    </article>
  );
};

export default Card;
```

ШАГ 4: СОЗДАНИЕ МОКОВЫХ ДАННЫХ

Объяснение: Моковые данные позволяют разрабатывать компоненты без подключения к реальному API. Мы создадим структуры, похожие на данные из ЛР6.

```
// src/data/mockData.js

// Объяснение: Эти данные имитируют ответы API из ЛР6
// Мы создаем разные типы данных для разных вариантов заданий

export const products = [
  {
    id: 1,
    title: 'Смартфон',
    description: 'Современный смартфон с отличной камерой',
    price: 29999,
    image: 'https://via.placeholder.com/300x200/007bff/ffffff?text=Smartphone',
    category: 'electronics',
    rating: 4.5,
    inStock: true,
  },
  {
    id: 2,
    title: 'Ноутбук',
    description: 'Мощный ноутбук для работы и игр',
    price: 59999,
    image: 'https://via.placeholder.com/300x200/28a745/ffffff?text=Laptop',
    category: 'electronics',
    rating: 4.8,
    inStock: true,
  },
  // Добавьте больше товаров по аналогии
];
```

```
export const news = [
  {
    id: 1,
    title: 'Новость о технологиях',
    description: 'Последние новости в мире технологий и инноваций',
    image: 'https://via.placeholder.com/300x200/17a2b8/ffffff?text=Tech+News',
    publishedAt: '2024-01-15T10:00:00Z',
    author: 'Техноблог',
    category: 'technology',
    views: 1500,
  },
  // Добавьте больше новостей по аналогии
];

// Данные для других типов приложений
export const tasks = [
  {
    id: 1,
    title: 'Изучить React',
    description: 'Освоить основы компонентного подхода',
    completed: false,
    priority: 'high',
    dueDate: '2024-02-01',
  },
];

export const events = [
  {
    id: 1,
    title: 'Встреча по проекту',
    description: 'Обсуждение этапов разработки',
    date: '2024-01-20T14:00:00Z',
    location: 'Конференц-зал',
    participants: 5,
  },
];
```

ШАГ 5: СОЗДАНИЕ КОМПОЗИТНЫХ КОМПОНЕНТОВ

Объяснение: Композитные компоненты объединяют несколько простых компонентов для создания сложных интерфейсов.

Компонент заголовка (`src/components/layout/Header.jsx`):

```
// Объяснение: Этот компонент демонстрирует:
// 1. Организацию layout-компонентов
// 2. Обработку событий
// 3. Передачу функций через props

const Header = ({ title, onSearch }) => {
  return (
    <header className='header'>
      <div className='header__container'>
        <h1 className='header__title'>{title}</h1>

        {/* Условный рендеринг поиска */}
        {onSearch && (
          <div className='header__search'>
            <input
              type='text'
              placeholder='Поиск...'
              onChange={(e) => onSearch(e.target.value)} // Передаем значение в родительский компонент
              className='header__search-input'
            />
          </div>
        )}
      </div>
    </header>
  );
};

export default Header;
```

Специализированный компонент продукта (`src/components/features/ProductCard.jsx`):

```
// Объяснение: Этот компонент показывает, как создавать специализированные компоненты
// на основе базовых, добавляя специфичную логику отображения

import Card from '../ui/Card';
import Button from '../ui/Button';

const ProductCard = ({
  product, // Объект продукта со всеми данными
  onAddToCart, // Функция для добавления в корзину
  onViewDetails, // Функция для просмотра деталей
}) => {
  return (
    <Card
      title={product.title}
      description={product.description}
      image={product.image}
      className='product-card'
    >
      {/* Мета-информация о продукте */}
      <div className='product-card__meta'>
        <span className='product-card__price'>${product.price}</span>
        <span className='product-card__category'>{product.category}</span>
        {product.rating && (
          <span className='product-card__rating'>★ {product.rating}</span>
        )}
      </div>

      {/* Кнопки действий */}
      <div className='product-card__actions'>
        <Button
          variant='primary'
          onClick={() => onAddToCart(product)} // Передаем весь объект продукта
        >
          В корзину
        </Button>
        <Button variant='secondary' onClick={() => onViewDetails(product)}>
          Подробнее
        </Button>
      </div>
    </Card>
  );
};

export default ProductCard;
```

ШАГ 6: КОМПОНЕНТ СПИСКА С ФИЛЬТРАЦИЕЙ

Объяснение: Этот компонент демонстрирует работу с массивами данных и фильтрацией на основе props.

```
// src/components/features/ProductList.jsx
import ProductCard from './ProductCard';

const ProductList = ({
  products, // Массив продуктов
  filterText = '', // Текст для фильтрации
  onAddToCart, // Обработчик добавления в корзину
  onViewDetails, // Обработчик просмотра деталей
}) => {
  // Объяснение: фильтрация происходит на основе props (без внутреннего состояния)
  // Это "контролируемый компонент" - вся логика управления в родительском компоненте
  const filteredProducts = products.filter(
    (product) =>
      product.title.toLowerCase().includes(filterText.toLowerCase()) ||
      product.description.toLowerCase().includes(filterText.toLowerCase()),
  );

  // Состояние пустого списка
  if (filteredProducts.length === 0) {
    return (
      <div className='product-list__empty'>
        {filterText
          ? 'Товары по вашему запросу не найдены'
          : 'Товары не найдены'}
      </div>
    );
  }

  return (
    <div className='product-list'>
      <div className='product-list__grid'>
        {/* Объяснение: map() преобразует массив данных в массив React-элементов */}
        {filteredProducts.map((product) => (
          <ProductCard
            key={product.id} // Важно: key помогает React отслеживать элементы
            product={product}
            onAddToCart={onAddToCart}
            onViewDetails={onViewDetails}
          />
        ))}
      </div>
    </div>
  );
};

export default ProductList;
```

ШАГ 7: ГЛАВНЫЙ КОМПОНЕНТ ПРИЛОЖЕНИЯ

Объяснение: App.jsx - корневой компонент, который объединяет все остальные компоненты.

```
// src/App.jsx
import { products, news } from './data/mockData';
import Header from './components/layout/Header';
import ProductList from './components/features/ProductList';
import Card from './components/ui/Card';
import './App.css';

function App() {
  // Объяснение: Эти функции - заглушки для обработчиков событий
  // В ЛР17 мы заменим их на реальную логику с состоянием

  const handleSearch = (searchTerm) => {
    console.log('Поиск:', searchTerm);
    // В ЛР17: здесь будет обновление состояния фильтра
  };

  const handleAddToCart = (product) => {
    console.log('Добавить в корзину:', product);
    // В ЛР17: здесь будет обновление состояния корзины
  };

  const handleViewDetails = (product) => {
    console.log('Просмотр деталей:', product);
    // В ЛР18: здесь будет навигация на страницу товара
  };

  return (
    <div className='app'>
      {/* Header компонент с props */}
      <Header title='Интернет-магазин' onSearch={handleSearch} />

      <main className='app__main'>
        <div className='app__container'>
          {/* Секция с товарами */}
          <section className='app__section'>
            <h2>Наши товары</h2>
            <ProductList
              products={products}
              filterText='' // В ЛР17: будет управляться из состояния
              onAddToCart={handleAddToCart}
              onViewDetails={handleViewDetails}
            />
          </section>

          {/* Секция с новостями */}
          <section className='app__section'>
            <h2>Последние новости</h2>
            <div className='news-grid'>
              {news.map((item) => (
                <Card
                  key={item.id}
                  title={item.title}
                  description={item.description}
                  image={item.image}
                >
                  <div className='news-meta'>
                    <span>Автор: {item.author}</span>
                    <span>
                      {new Date(item.publishedAt).toLocaleDateString()}
                    </span>
                  </div>
                </Card>
              ))}
            </div>
          </section>

          {/* Теоретический блок */}
          <section className='app__section theory-section'>
            <h2>Концепции компонентного подхода</h2>

            <div className='theory-grid'>
              <Card
                title='SPA (Single Page Application)'
                className='theory-card'
              >
                <p>
                  <strong>Одностраничное приложение</strong> - веб-приложение,
                  которое работает в рамках одной HTML-страницы, динамически
                  обновляя контент вместо загрузки новых страниц.
                </p>
                <ul>
                  <li> Быстрая реакция на действия пользователя</li>
                  <li> Уменьшение нагрузки на сервер</li>
                  <li> Более плавный пользовательский опыт</li>
                  <li> Возможность оффлайн-работы (с PWA)</li>
                </ul>
              </Card>

              <Card title='Virtual DOM' className='theory-card'>
                <p>
                  <strong>Виртуальный DOM</strong> - легковесная копия реального
                  DOM, которая позволяет React эффективно обновлять интерфейс.
                </p>
                <ul>
                  <li> Высокая производительность</li>
                  <li> Автоматическая оптимизация обновлений</li>
                  <li> Упрощение разработки сложных интерфейсов</li>
                  <li> Сравнение и пакетное обновление изменений</li>
                </ul>
              </Card>

              <Card title='Сравнение фреймворков' className='theory-card'>
                <div className='framework-comparison'>
                  <div className='framework'>
                    <h4> React</h4>
                    <ul>
                      <li>
                        <strong>Тип:</strong> Библиотека для UI
                      </li>
                      <li>
                        <strong>Поток данных:</strong> Однонаправленный
                      </li>
                      <li>
                        <strong>Кривая обучения:</strong> Средняя
                      </li>
                      <li>
                        <strong>Популярность:</strong> Очень высокая
                      </li>
                    </ul>
                  </div>
                  <div className='framework'>
                    <h4> Vue</h4>
                    <ul>
                      <li>
                        <strong>Тип:</strong> Прогрессивный фреймворк
                      </li>
                      <li>
                        <strong>Поток данных:</strong> Двусторонний
                      </li>
                      <li>
                        <strong>Кривая обучения:</strong> Низкая
                      </li>
                      <li>
                        <strong>Популярность:</strong> Растущая
                      </li>
                    </ul>
                  </div>
                  <div className='framework'>
                    <h4> Angular</h4>
                    <ul>
                      <li>
                        <strong>Тип:</strong> Полноценный фреймворк
                      </li>
                      <li>
                        <strong>Поток данных:</strong> Двусторонний
                      </li>
                      <li>
                        <strong>Кривая обучения:</strong> Высокая
                      </li>
                      <li>
                        <strong>Популярность:</strong> Корпоративная
                      </li>
                    </ul>
                  </div>
                </div>
              </Card>
            </div>
          </section>
        </div>
      </main>
    </div>
  );
}

export default App;
```

ШАГ 8: ДОБАВЛЕНИЕ СТИЛЕЙ

Объяснение: Стили помогают сделать интерфейс привлекательным. Мы используем CSS модули для изоляции стилей.

```
/* src/App.css */

/* Базовые стили */
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  line-height: 1.6;
  color: #333;
}

.app {
  min-height: 100vh;
  background-color: #f8f9fa;
}

.app__container {
  max-width: 1200px;
  margin: 0 auto;
  padding: 0 20px;
}

.app__section {
  margin-bottom: 40px;
}

.app__section h2 {
  margin-bottom: 20px;
  color: #2c3e50;
  font-size: 1.8rem;
  border-bottom: 2px solid #3498db;
  padding-bottom: 10px;
}

/* Header */
.header {
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  color: white;
  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
  padding: 1.5rem 0;
  margin-bottom: 2rem;
}

.header__container {
  max-width: 1200px;
  margin: 0 auto;
  padding: 0 20px;
  display: flex;
  justify-content: space-between;
  align-items: center;
  flex-wrap: wrap;
  gap: 20px;
}

.header__title {
  font-size: 2rem;
  font-weight: 700;
}

.header__search-input {
  padding: 12px 16px;
  border: none;
  border-radius: 25px;
  width: 300px;
  font-size: 16px;
  box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
}

.header__search-input:focus {
  outline: none;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.2);
}

/* Cards */
.card {
  background: white;
  border-radius: 12px;
  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
  overflow: hidden;
  transition: transform 0.3s ease, box-shadow 0.3s ease;
  height: 100%;
}

.card:hover {
  transform: translateY(-5px);
  box-shadow: 0 8px 15px rgba(0, 0, 0, 0.15);
}

.card__image img {
  width: 100%;
  height: 200px;
  object-fit: cover;
  transition: transform 0.3s ease;
}

.card:hover .card__image img {
  transform: scale(1.05);
}

.card__content {
  padding: 24px;
}

.card__title {
  margin: 0 0 12px 0;
  font-size: 1.4rem;
  color: #2c3e50;
  font-weight: 600;
}

.card__description {
  color: #7f8c8d;
  margin-bottom: 16px;
  line-height: 1.5;
}

/* Buttons */
.btn {
  padding: 12px 24px;
  border: none;
  border-radius: 8px;
  cursor: pointer;
  font-size: 14px;
  font-weight: 600;
  transition: all 0.3s ease;
  text-transform: uppercase;
  letter-spacing: 0.5px;
}

.btn--primary {
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  color: white;
}

.btn--primary:hover {
  transform: translateY(-2px);
  box-shadow: 0 4px 8px rgba(102, 126, 234, 0.4);
}

.btn--secondary {
  background: #95a5a6;
  color: white;
}

.btn--secondary:hover {
  background: #7f8c8d;
  transform: translateY(-2px);
}

.btn:disabled {
  background: #bdc3c7;
  cursor: not-allowed;
  transform: none;
}

/* Product List */
.product-list__grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(320px, 1fr));
  gap: 24px;
}

.product-list__empty {
  text-align: center;
  padding: 60px 20px;
  color: #7f8c8d;
  font-size: 1.2rem;
}

.product-card__meta {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 20px;
  padding: 12px 0;
  border-top: 1px solid #ecf0f1;
  border-bottom: 1px solid #ecf0f1;
}

.product-card__price {
  font-size: 1.4rem;
  font-weight: 700;
  color: #27ae60;
}

.product-card__category {
  background: #3498db;
  color: white;
  padding: 4px 12px;
  border-radius: 20px;
  font-size: 0.8rem;
  font-weight: 600;
}

.product-card__rating {
  color: #f39c12;
  font-weight: 600;
}

.product-card__actions {
  display: flex;
  gap: 12px;
}

.product-card__actions .btn {
  flex: 1;
}

/* News */
.news-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
  gap: 24px;
}

.news-meta {
  display: flex;
  justify-content: space-between;
  font-size: 0.9rem;
  color: #7f8c8d;
  margin-top: 12px;
  padding-top: 12px;
  border-top: 1px solid #ecf0f1;
}

/* Theory Section */
.theory-section {
  background: white;
  padding: 40px;
  border-radius: 12px;
  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
}

.theory-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(350px, 1fr));
  gap: 24px;
}

.theory-card {
  border: 2px solid #3498db;
}

.theory-card .card__title {
  color: #3498db;
  font-size: 1.3rem;
}

.framework-comparison {
  display: flex;
  flex-direction: column;
  gap: 20px;
}

.framework h4 {
  margin: 0 0 12px 0;
  color: #2c3e50;
  font-size: 1.1rem;
}

.framework ul {
  list-style: none;
  padding: 0;
}

.framework li {
  padding: 4px 0;
  border-bottom: 1px solid #ecf0f1;
}

.framework li:last-child {
  border-bottom: none;
}

/* Адаптивность */
@media (max-width: 768px) {
  .header__container {
    flex-direction: column;
    text-align: center;
  }

  .header__search-input {
    width: 100%;
    max-width: 400px;
  }

  .product-list__grid {
    grid-template-columns: 1fr;
  }

  .theory-grid {
    grid-template-columns: 1fr;
  }

  .app__container {
    padding: 0 15px;
  }
}
```

ВАРИАНТЫ ЗАДАНИЙ ДЛЯ СТУДЕНТОВ

ОБЩЕЕ ЗАДАНИЕ ДЛЯ ВСЕХ ВАРИАНТОВ:

Создайте React-приложение согласно вашему варианту. **Для выбора номера варианта для бригады используется Генератор вариантов.**

Реализуйте:

- Минимум 5 различных компонентов
- Композитные компоненты, объединяющие базовые
- Моковые данные, соответствующие тематике
- Обработчики событий (заглушки)
- Стилизацию компонентов

---

1 ПОДГРУППА (Варианты 1-8)

Вариант 1: Новостной портал "NewsHub"

- **Компоненты:** NewsCard, NewsList, NewsFilters, NewsHeader, CategoryTabs
- **Данные:** статьи с заголовком, текстом, автором, датой, категорией
- **Особенности:** фильтрация по категориям, поиск по тексту

Вариант 2: Интернет-магазин "TechStore"

- **Компоненты:** ProductCard, ProductGrid, ShoppingCart, PriceFilter, Rating
- **Данные:** товары с ценой, рейтингом, описанием, изображением
- **Особенности:** корзина покупок, фильтрация по цене и рейтингу

Вариант 3: Задачник "TaskManager"

- **Компоненты:** TaskItem, TaskList, AddTaskForm, FilterButtons, PriorityBadge
- **Данные:** задачи с названием, описанием, приоритетом, статусом
- **Особенности:** фильтрация по статусу и приоритету, добавление задач

Вариант 4: Блог-платформа "BlogSpace"

- **Компоненты:** PostCard, PostList, AuthorInfo, TagList, CommentPreview
- **Данные:** посты с заголовком, содержанием, автором, тегами, датой
- **Особенности:** фильтрация по тегам, поиск по содержанию

Вариант 5: Каталог фильмов "MovieCatalog"

- **Компоненты:** MovieCard, MovieGrid, GenreFilter, SearchBox, RatingStars
- **Данные:** фильмы с названием, описанием, жанром, рейтингом, годом
- **Особенности:** поиск по названию, фильтрация по жанру и рейтингу

Вариант 6: Погодное приложение "WeatherApp"

- **Компоненты:** WeatherCard, ForecastList, CitySearch, WeatherIcon, Temperature
- **Данные:** прогноз погоды с температурой, описанием, иконкой, влажностью
- **Особенности:** поиск по городам, отображение текущей погоды и прогноза

Вариант 7: Социальная сеть "SocialNetwork"

- **Компоненты:** UserProfile, PostCard, FriendList, LikeButton, ShareButton
- **Данные:** пользователи, посты, комментарии, лайки
- **Особенности:** лента новостей, профиль пользователя, социальные действия

Вариант 8: Образовательная платформа "EduPlatform"

- **Компоненты:** CourseCard, LessonList, ProgressBar, InstructorInfo, Review
- **Данные:** курсы, уроки, преподаватели, отзывы, прогресс
- **Особенности:** отображение прогресса, фильтрация курсов, рейтинги

---

2 ПОДГРУППА (Варианты 9-16)

Вариант 9: Ресторанное приложение "FoodDelivery"

- **Компоненты:** DishCard, RestaurantInfo, CartItem, CategoryMenu, Rating
- **Данные:** блюда, рестораны, категории, цены, отзывы
- **Особенности:** корзина заказов, фильтрация по категориям и цене

Вариант 10: Библиотека "BookLibrary"

- **Компоненты:** BookCard, AuthorCard, SearchBar, GenreFilter, BookStatus
- **Данные:** книги, авторы, жанры, доступность, рейтинги
- **Особенности:** поиск по автору и названию, фильтрация по жанру

Вариант 11: Фитнес-трекер "FitTrack"

- **Компоненты:** WorkoutCard, ExerciseItem, ProgressChart, Timer, Achievement
- **Данные:** тренировки, упражнения, прогресс, достижения
- **Особенности:** таймер тренировок, графики прогресса, достижения

Вариант 12: Портфолио "PhotoGallery"

- **Компоненты:** PhotoCard, AlbumGrid, FilterButtons, Lightbox, PhotoInfo
- **Данные:** фотографии, альбомы, теги, метаданные
- **Особенности:** фильтрация по тегам, просмотр в лайтбоксе

Вариант 13: Путешествия "TravelPlanner"

- **Компоненты:** DestinationCard, TripItinerary, BookingForm, MapPreview, Review
- **Данные:** направления, маршруты, отели, отзывы, цены
- **Особенности:** планирование маршрута, форма бронирования

Вариант 14: Музыкальный плеер "MusicStream"

- **Компоненты:** SongCard, Playlist, PlayerControls, VolumeSlider, ProgressBar
- **Данные:** треки, альбомы, исполнители, плейлисты
- **Особенности:** элементы управления плеером, прогресс воспроизведения

Вариант 15: Финтех "BudgetTracker"

- **Компоненты:** TransactionItem, BudgetCategory, Chart, AddTransaction, Filter
- **Данные:** транзакции, категории, бюджет, графики
- **Особенности:** фильтрация по дате и категории, добавление транзакций

Вариант 16: Ивенты "EventManager"

- **Компоненты:** EventCard, CalendarView, RegistrationForm, LocationMap, Attendee
- **Данные:** события, даты, локации, участники, билеты
- **Особенности:** календарь событий, форма регистрации

---

6. КОНТРОЛЬНЫЕ ВОПРОСЫ

1. **Что такое JSX и чем он отличается от обычного HTML?** Приведите 3 конкретных примера различий из вашего проекта.
2. **Как работает передача данных между компонентами через props?** Покажите на примере передачи данных от родительского компонента к дочернему в вашем проекте.
3. **В чем преимущества компонентного подхода** по сравнению с традиционной разработкой на чистом JavaScript? Приведите примеры из вашего проекта.
4. **Объясните концепцию Virtual DOM** и ее преимущества для производительности веб-приложений.
5. **Сравните React, Vue и Angular** по следующим критериям: кривая обучения, производительность, экосистема, области применения.

7. СОДЕРЖАНИЕ ОТЧЕТА

- **Титульный лист**
- **Цель работы**
- **Краткие теоретические сведения:**
  - Компонентный подход в React
  - JSX и его особенности (отличия от HTML)
  - Концепция SPA и Virtual DOM
  - Сравнение фреймворков (React vs Vue vs Angular)
- **Ход работы:**
  - Описание созданных компонентов и их назначения
  - Примеры использования JSX и props
  - Скриншоты работающего приложения
  - Исходный код ключевых компонентов
  - Ссылка на репозиторий GitHub
- **Ответы на контрольные вопросы**
- **Выводы** (что узнали, какие навыки приобрели)

ВАЖНЫЕ ЗАМЕЧАНИЯ

1. **Не используйте useState и useEffect** - эти темы будут в следующем семестре.
2. **Все данные должны быть статическими** (моковые данные)
3. **Фокусируйтесь на понимании JSX и props**
4. **Компоненты должны быть переиспользуемыми**
5. **Подготовьте базу для следующих лабораторных работ**

Добавить ответ на задание

Состояние ответа

Номер попытки	Попытка 1.
Состояние ответа на задание	Ответы на задание еще не представлены
Состояние оценивания	Не оценено
Оставшееся время	3 дн. 7 час. осталось
Последнее изменение	-
Комментарии к ответу	› Комментарии (0)